



ADVANCED PROGRAMMING

CAMPUS LOST ITEMS

Mazen Thobian

320250329

The Problem :



Many items get lost by us when we are on campus, and it's not easy to find those items.

The Solution :



A networked Java desktop application that allows us to report lost or found items using an automated matching algorithm to evaluate item attributes.

The System Architecture :

Client-Server Model :



A central **client/server** architecture is employed by this software application in order to ensure proper communication.

MatcherServer acts as the control mechanism, constantly monitoring Port **7500** for connections.

When a student uploads his report, the client program (**The GUI**) directly connects with the server through a bi-directional socket connection based on **TCP/IP**.

With everything kept centralised, the server is able to take care of important activities like matching algorithms, ID generation, and maintaining a database in a safe manner.

Data Serialization :



Object Serialization provided by Java is used in the system to facilitate the smooth interaction between the client **GUIs** and the server.

The classes such as **CampusItem**, **IDCard**, and **NetworkRequest** implement the **Serializable interface**, which enables the transformation of those objects into byte streams that can be transported via the network connection.

Once the user interacts with the elements on the screen, the corresponding actions with the associated data are **encapsulated** into the **NetworkRequest** and sent via the **ObjectOutputStream** to the server, which then de-serializes the input into the corresponding Java objects.

Multithreading :



We need a system to accommodate many users simultaneously without affecting performance.

However, if the server attended to the users sequentially, then any delay experienced by one user would affect all other users, so that, the server is **multithreaded**.

In case the **MatcherServer** gets a connection request from any client, then a **Thread** is created for the **ClientHandler** to attend to that user.

By that, the server continues to listen to incoming connections and accepts any incoming connection immediately without delay.

Consequently, many users can submit their reports and receive matches at the same time.

The Matching Algorithm :

Moving Beyond Binary Evaluation :



In normal databases, **if/else** comparisons like "**this.location.equals(other.location)**" may cause problems when customers make spelling errors or use different words (e.g. "**Main Library**" instead of "**Library**").

That's why, the system relies on a **scoring mechanism** instead of an exact comparison approach.

A point system is used where each characteristic is assigned points depending on its accuracy in the real world.

These points are added up, and the probability of a match is calculated. The minimum threshold for a match is set to **80** points.

Modular Object-Oriented Scoring :



The scoring system is **object-oriented** because:

Score is calculated in the **CampusItem** class, which handles shared attributes. Subclasses like **IDCard**, **ElectronicDevice** or **Jewelry**, **override** the **calculateMatchScore()** method:

- They first call the **superclass** method to get baseline points.
- Then they add extra checks for their own unique fields.

This design made it easy to add new categories with custom scoring rules without changing the core system.

Strict vs. Forgiving Evaluation Metrics :



The algorithm uses two kinds of tests to simulate the decision-making :

Definitive Tests (Strict Matching)

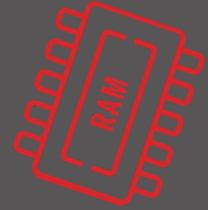
Unique identifiers like **StudentID** numbers are considered to be definitive. In case of an exact match in this category, you get **100 points** without taking anything else into account.

Fuzzy Tests (Forgiving Matching)

For fields like **names** and **locations**, inputs are first normalized (**converted to lower case**) and compared using substring matching. “**Ahmed**” matches “**Ahmed Ali**”, Points can be accumulated (**for instance, +80** for a name match, **+10** for a location match).

Technical Challenges & Solutions :

1. Background threads were updating their local memory of the item lists, but new connections couldn't see the new data.



Solution :

Implemented Java's

`Collections.synchronizedList()` to wrap the **`ArrayLists`**, ensuring thread-safe, real-time memory visibility across the entire server.

2. Extracting string data from Java Swing JTextFields and ensuring it perfectly mapped to the superclass constructors without variable drift or `NullPointerExceptions`.

Solution :

Enforced strict constructor parameters and implemented **`.trim()`**, **`null-checks`** before running **`.toLowerCase()`** and **`.contains()`** string evaluations.

Data Persistence and State Recovery:

Survival of data beyond the lifecycle of the servers is ensured by having a centralized **DatabaseManager** that has persistence based on files.



Instead of depending on any **external SQL databases**, this program has used the built-in functionalities of **Java's I/O streams** and **object serialization**.

Models such as **CampusItem** and **IDCard** are implemented with the use of the **serializable interface**, which enables the writing of objects directly into **.dat** files.

With each addition of update to a new item, the list is synchronized with the filesystem. The server would automatically initialize itself from these **.dat** files upon startup.

This database is portable and powerful enough to preserve all information even during crashes and restarts of the server.

Features :

1. Smart Matching System.



2. Real-Time Multithreading.



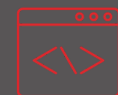
3. Easy Data Saving.



4. Strong Client-Server Design.



5. Simple User Interface.



Evaluation :



Solves a practical / real-world problem	
Proper use of classes and objects	
Encapsulation	
Inheritance	
Polymorphism	
Interfaces & Abstract classes	
Code organization and modularity	
Correct client-server communication using sockets	
File Handling and Data Structures	
Exception Handling	
Clear project explanation and workflow	
Code readability and comments	
Proper documentation / report structure	
Multi-Threading (bonus)	



```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Thank you!");  
    }  
}
```

Thank You!